

INSTALL & RUN

```
python -m venv .venv && pip install flask # or uv add
pip install "flask[async]" # extras: async, dotenv
from flask import Flask # hello.py
app = Flask(__name__)
@app.get("/")
def index(): return "<p>Hi</p>"
flask --app hello run --debug # reloader + debugger
flask --app hello run --host 0.0.0.0 --port 8080
.flaskenv (committed) and .env (secrets) via python-dotenv:
FLASK_APP=hello, FLASK_DEBUG=1; --env-file picks another.
macOS: AirPlay owns port 5000. Dev server and debugger: never in production.
```

APPLICATION FACTORY & LAYOUT

```
def create_app(config: dict | None = None) -> Flask:
    app = Flask(__name__, instance_relative_config=True)
    app.config.from_object("bookshelf.config.Dev")
    app.config.from_pyfile("config.py", silent=True)
    app.config.update(config or {}) # tests
    db.init_app(app); login.init_app(app) # extensions.py
    app.register_blueprint(catalog_bp)
    return app
flask --app bookshelf run # or 'bookshelf:create_app("dev")'
pyproject.toml # pip install -e .
src/bookshelf/ __init__.py config.py extensions.py models/
catalog/ auth/ admin/ api/ templates/ static/
migrations/ tests/ instance/
```

CONFIG LOADERS & KEY SETTINGS

```
app.config.from_object(Cls) .from_pyfile("c.py")
app.mapping(A=1) .from_envvar("APP_CFG")
.from_prefixed_env() # FLASK_SECRET_KEY, FLASK_A_B
.from_file("c.toml", load=tomllib.load, text=False)
SECRET_KEY None; token_hex() from env
MAX_CONTENT_LENGTH None -> 413 when set
MAX_FORM_MEMORY_SIZE 500000 MAX_FORM_PARTS 1000
PERMANENT_SESSION_LIFETIME 31 days
SERVER_NAME, PREFERRED_URL_SCHEME None, http
TESTING, PROPAGATE_EXCEPTIONS False, None
SEND_FILE_MAX_AGE_DEFAULT None - USE_X_SENDFILE False
```

CONTEXTS & REQUEST LIFECYCLE

```
current_app, g (app ctx), request, session (request ctx):
LocalProxy over contextvars.
with app.app_context(): db.create_all() # scripts, CLI
with app.test_request_context("/x/"): ... # tests
Order: wsgi_app -> push app + request ctx, open session, match URL
-> request_started -> url_value_preprocessor -> before_request (a return value ends it) -> view or errorhandler ->
after_this_request -> after_request -> save session -> request_finished -> teardown_request -> pop request ctx ->
teardown_appcontext -> pop app ctx. Streamed bodies run after
teardown: stream_with_context. 3.2 merges the two contexts.
```

ROUTING, CONVERTERS & URL FOR

```
@app.route("/books/<int:id>", methods=["GET", "POST"])
@app.get/post/out/patch/delete("/books/")
app.add_url_rule("/x", endpoint="x", view_func=f)
# converters: string int float path uuid
class IsbnConverter(BaseConverter):
    regex = r"^\d{13}$" # to python / to url
app.url_map.converters["isbn"] = IsbnConverter
url_for("catalog.show", id=1, _external=True, _scheme="https")
url_for(".show") url_for("static", filename="app.css")
strict_slashes=False; subdomain=: host_matching;
@url_value_preprocessor + @url_defaults for /<lang>/;
automatic OPTIONS/HEAD; flask routes.
```

REQUEST DATA

```
request.args.get("page", 1, type=int) # .getlist("tag")
request.form["t"].values .files["cover"].data
request.get_json() # 415 not JSON, 400 malformed
request.headers["X-Id"] request.cookies.get("c") .is_json
request.method .path .url .remote_addr # needs ProxyFix
Limits: MAX_CONTENT_LENGTH (413), MAX_FORM_MEMORY_SIZE,
MAX_FORM_PARTS; per request request.max_content_length (3.1).
```

RESPONSES & JSON

```
return "text" | {"id": 1} | [1, 2] # dict/list = JSON
return body, 201, {"Location": url} # generator streams
resp = make_response(render_template("a.html"), 200)
resp.set_cookie("x", "y", httponly=True, secure=True,
               samesite="lax", max_age=3600)
return redirect(url_for("catalog.index")) # 302; 303 in 3.2
abort(404) jsonify(items=rows) # app.json provider
send_file(p, download_name="b.pdf", as_attachment=True)
send_from_directory(app.config["UPLOADS"], name)
app.json.sort_keys = False; custom provider or flask-orjson.
```

BLUEPRINTS

```
bp = Blueprint("catalog", __name__, url_prefix="/books",
               template_folder="templates")
app.register_blueprint(bp, url_prefix="/b", name="alt")
parent.register_blueprint(child) # nested (2.0)
@bp.before_app_request @bp.errorhandler(404)
@bp.app_context_processor @bp.cli.command("x")
Endpoint is catalog.show; setup after the first request is an error (2.3).
```

ERRORS, HOOKS & CLASS-BASED VIEWS

```
@app.errorhandler(404) / (HTTPException) / (BookError)
def handle(e): # RFC 9457 problem details
    return ({ "title": e.name, "status": e.code, e.code,
              "Content-Type": "application/problem+json" })
@app.app_errorhandler(404) # a bp handler never sees 404/405
@app.before_request # a return value short-circuits
@app.after_request # must return the response
@app.teardown_request # + teardown_appcontext
class Books(MethodView): # get / post / ...
    decorators = [login_required]
    app.add_url_rule("/books", view_func=Books.as_view("books"))
```

JINJA ESSENTIALS

```
{{ book.title }} {# autoescaped: .html .htm .xml .svg #}
{{ html[safe] }} Markup(x) {# trusted text only #}
{% extends "base.html" %} {% block body %}{% endblock %}
{% include "_nav.html" %} {% from "m.html" import field %}
{# for c, m in get_flashed_messages(with_categories=true) %}
@app.template_filter() @app.template_test()
@app.template_global() @app.context_processor
render_template("a.html", **ctx) stream_template("big.html")
Context: config request session g url_for; macros: {% macro
f(x) %}.
```

SESSIONS, COOKIES & FLASHING

```
session["uid"] = user.id # signed, NOT encrypted
session.permanent = True # PERMANENT_SESSION_LIFETIME
SECRET_KEY = os.environ["SECRET_KEY"] # token_hex(32)
SECRET_KEY_FALLBACKS = [old_key] # rotation (3.1)
flash("Saved", "success")
get_flashed_messages(with_categories=True)
SESSION_COOKIE_NAME, DOMAIN, PATH, HTTPONLY, SECURE,
SAMESITE, PARTITIONED; Vary: Cookie. Server side: Flask-Session
(Redis).
```

FLASK-WTF & CSRF

```
class BookForm(FlaskForm):
    title = StringField(validators=[DataRequired()])
    cover = FileField(validators=[FileAllowed(["png"])])
if form.validate_on_submit(): ... # POST + valid + CSRF
{{ form.hidden_tag() }} {{ form.title.errors }}
csrf = CSRFProtect(app) # all unsafe methods
<meta name="csrf-token" content="{% csrf_token() %}">
fetch(url, headers: {"X-CSRFToken": token}) # @csrf.exempt
WTF_CSRF_TIME_LIMIT 3600 s; custom validate_title().
```

AUTH: FLASK-LOGIN & FLASK-JWT-EXTENDED

```
login = LoginManager(app); login.login_view = "auth.login"
class User(UserMixin, db.Model): ...
@login.user_loader
def load(uid): return db.session.get(User, int(uid))
login.user_loader remember=True logout_user() current_user
@login_required @fresh_login_required
generate_password_hash(pw) # script; check_password_hash
jwt = JWTManager(app) # JWT_SECRET_KEY from env
create_access_token(identity=str(u.id)) # string identity
@jwt_required() @jwt_refresh_from_request get_jwt()
@jwt.token_in_blocklist_loader # revocation
JWT_TOKEN_LOCATION = ["headers", "cookies"] # cookies: CSRF
```

SECURITY SETTINGS

```
SECRET_KEY_FALLBACKS random, from env; rotate
TRUSTED_HOSTS ["bookshelf.example"] (3.1)
SESSION_COOKIE_* SECURE, HTTPONLY, SAMESITE="Lax"
MAX_* CONTENT_LENGTH, FORM_MEMORY_SIZE, FORM_PARTS
Keep autoescape; never Markup(user_input); quote attributes. CSP,
HSTS, X-Content-Type-Options via Flask-Talisman; CORS via Flask-
CORS. secure_filename + size limit for uploads. pip-audit. No
debugger in production.
```

FLASK-SQALCHEMY 2.0 STYLE

```
class Base(DeclarativeBase): pass
db = SQLAlchemy(model_class=Base) # SQLALCHEMY_DATABASE_URI
class Book(db.Model):
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    author_id: Mapped[int] = mapped_column(
        ForeignKey("author.id"))
    author: Mapped["Author"] = relationship(backref="books")
stmt = db.select(Book).where(Book.year > 2000)
rows = db.session.execute(stmt).scalars().all()
db.session.get(Book, 1) db.get_or_404(Book, id)
db.first_or_404(stmt) db.one_or_404(stmt)
page = db.paginate(stmt, per_page=20, max_per_page=100)
page.items .pages .has_next .next_num .page.iter_pages()
db.session.add(b); db.session.commit() db.create_all()
One session per app context. SQLALCHEMY_BINDS.
```

FLASK-MIGRATE (ALEMBIC)

```
migrate = Migrate(app, db)
flask db init # once: migrations/
flask db migrate -m "add isbn" # autogenerate, then REVIEW
flask db upgrade flask db downgrade -1
flask db current | history | heads | stamp | merge
Autogenerate misses renames; SQLite needs render_as_batch. Run
upgrade as a release step or K8s Job.
```

FLASK-SMOREST & OPENAPI

```
app.config.update(API_TITLE="Bookshelf", API_VERSION="v1",
                  OPENAPI_VERSION="3.1.0", OPENAPI_URL_PREFIX="/")
api = Api(app)
blp = Blueprint("books", __name__, url_prefix="/api/v1/books")
@blp.route("/") # outermost
class Books(MethodView):
    @blp.arguments(QuerySchema, location="query") # 422
    @blp.response(200, BookSchema(many=True)) # status first
    @blp.paginate()
    def get(self, args, pagination_parameters): ...
    @blp.arguments(BookSchema)
    @blp.response(201, BookSchema)
    @blp.alt_response(409, schema=ErrorSchema)
    def post(self, data): ...
api.register_blueprint(blp)
Alternatives: APiFlask, flask-openapi3. Legacy: Flask-RESTX.
```

CLI & SIGNALS

```
@app.cli.command("seed") # no @with_appcontext since 2.2
@click.argument("n", type=int)
@click.option("--reset", is_flag=True)
def seed(n, reset): ... # flask --app bookshelf seed 10
books = AppGroup("books"); app.cli.add_command(books)
runner = app.test_cli_runner() # runner.invoke(args=...)
flask routes | shell @app.shell_context_processor
sig = Namespace().signal("view-added"); sig.send(app)
@sig.connect @sig.connect_via(app) sig.connect(to(f)
# request_started request_finished template_rendered ...
```

CELERY PATTERN (OFFICIAL)

```
def celery_init_app(app: Flask) -> Celery:
    class FlaskTask(Task):
        def __call__(self, *a, **kw):
            with app.app_context():
                return self.run(*a, **kw)
    c = Celery(app.name, task_cls=FlaskTask)
    c.config_from_object(app.config["CELERY"])
    c.set_default(); app.extensions["celery"] = c; return c
app.config.from_mapping(CELERY=dict(
    broker_url="redis://localhost",
    @shared_task(ignore_result=False)
    def digest(n): ... digest.delay(5) # .get()
    celery -A make_celery_worker --loglevel INFO
Not threads. RQ is simpler; Flask-Mail from a task.
```

ASYNC, GEVENT & QUART

```
pip install "flask[async]" # async def views and hooks
@app.get("/a")
async def a(): return await fetch() # loop per request
gunicorn -k gevent -w 2 "bookshelf:create_app()"
asgi_app = WsgiToAsgi(app) # asgiref; run on hypercorn
from quart import Quart; await request.get_json()
Async views are not faster: a worker per request, no background tasks.
Quart 0.23 (Python 3.13+) for websockets.
```

STREAMING & SSE

```
def rows():
    yield "id,title\n"
    for b in books: yield f"{b.id},{b.title}\n"
return Response(stream_with_context(rows()),
               mimetype="text/csv")
def events():
    while True: yield f"event: review\n{data: {js}\n\n"
return Response(events(), mimetype="text/event-stream",
               headers={"Cache-Control": "no-cache"})
Each SSE client holds a worker: gevent or Quart. flask-sock, Flask-
SocketIO for WebSockets.
```

PYTEST FIXTURES & TESTING

```
@pytest.fixture
def app(tmp_path):
    uri = f"sqlite:/// {tmp_path}/t.db"
    app = create_app({"TESTING": True,
                     "SQLALCHEMY_DATABASE_URI": uri})
    with app.app_context(): db.create_all()
    yield app
@pytest.fixture
def client(app): return app.test_client()
r = client.post("/b", json={"t": 1}, follow_redirects=True)
r.status_code r.json r.text r.history
with client.session_transaction() as s: s["uid"] = 1
monkeypatch, pytest-mock, factory_boy, coverage; Schemathesis on
the OpenAPI.
```

PRODUCTION: GUNICORN, WAITRESS, NGINX, PROXYFIX

```
gunicorn -w 4 -b 127.0.0.1:8080 "bookshelf:create_app()"
waitress-serve --call "bookshelf:create_app" # Windows
app.wsgi_app = ProxyFix(app.wsgi_app, x_for=1, x_proto=1,
                        x_host=1) # = trusted proxy hops
location / { proxy_pass http://127.0.0.1:8080;
  proxy_set_header Host $host;
  proxy_set_header X-Forwarded-Proto $scheme;
  proxy_set_header X-Forwarded-For
    $proxy_add_x_forwarded_for; }
location /static/ { alias /srv/bookshelf/static; }
# systemd/Supervisor; Docker non-root; migrations as a Job
```

CHANGED SINCE OLDER TUTORIALS & BOOKS

- 2.0** async def views, nested blueprints, @app.get, from_file; attachment_filename -> download_name
- 2.2** flask --app x run --debug; JSON provider app.json; contextvars; FLASK_ENV deprecated
- 2.3** FLASK_ENV, before_first_request, json_encoder removed; Blinker required; Vary: Cookie
- 3.0** flask.Markup/escape gone (use markupsafe); Werkzeug 3: script hashes; Flask-SQLAlchemy 3 (Model.query legacy)

- 3.1** SECRET_KEY_FALLBACKS, TRUSTED_HOSTS, SESSION_COOKIE_PARTITIONED, MAX_FORM_PARTS; Python 3.8 dropped
- 3.2 (unreleased)** contexts merged; redirect 303; __version__ removed; 3.9/3.10 dropped
- Model.query.get(1) -> db.session.get(Book, 1) .filter_by().all() -> db.session.execute(db.select(...)).scalars()
- Book.query.paginate(1, 20) -> db.paginate(stmt, page=1, per_page=20) (keyword-only)

- db.Column(db.String) -> Mapped[str] = mapped_column() with DeclarativeBase
- @app.before_first_request -> the application factory .app.run(debug=True) -> Gunicorn
- Flask-Script / manage.py -> flask CLI + Click .nosetests -> pytest .setup.py -> pyproject.toml
- identity=user.id (int) -> str(user.id) in Flask-JWT-Extended
- flask.__version__ -> importlib.metadata